1927
USTP
Southern
UNIVERSITY OF
SCIENCE AND TECHNOLOGY
OF SOUTHERN PHILIPPINES
Alubijid | Cagayan de Oro | Claveria | Jasaan | Oroquieta | Panaon

DATA STRUCTURES AND ALGORITHMS

Snake Game

Presented by:
Miji, Tion, Tubal

BSIT - IT1R4

DESCRIPTION OF THE SYSTEM

Our project is an interactive Snake Game developed using:

- C Programming Language
- Raylib Library

The system includes:

- Snake movement
- Food spawning
- Collision detection
- Input handling
- Wall Spawning

The main purpose of the project is to apply Data Structures and Algorithms concepts in an actual game system.

THE MAINLOOP

```c
int main(void)
{
    //Window
    SetConfigFlags(FLAG_WINDOW_TOPMOST);
    InitWindow(2* OFFSET + CELL_SIZE*CELL_COUNT, 2* OFFSET + CELL_SIZE*CELL_COUNT, "DSA Snake Game");
    InitAudioDevice();

    //Music
    bgm = LoadMusicStream("Snake Resources/Music/Ts Pmo.mp3");
    eatSFX = LoadSound("Snake Resources/Sfx/Snake Eat Food.wav");
    crashSFX = LoadSound("Snake Resources/Sfx/Crash.wav");
    slowSFX = LoadSound("Snake Resources/Sfx/Za Applo.wav");
    fastSFX = LoadSound("Snake Resources/Sfx/Food In Heaven.wav");
    wallWarning = LoadSound("Snake Resources/Sfx/Wall Warning.mp3");
    reverseSFX = LoadSound("Snake Resources/Sfx/Boom.mp3");
    PlayMusicStream(bgm);

    SetWindowFocused();
    SetTargetFPS(60);

    Snake snake = MakeSnake();
    wall = MakePattern();
    struct FoodNode* head = NULL;
    Init_Input(&Input);
    for(int i = 0; i < 4; i++){
        SpawnFood(&head, &snake, &wall);
    }
```

INITIALIZATION

- In the Mainloop, we initialize the window and music.
- Then we initialized the snake, walls, the food, input and spawn food 4x.

THE MAINLOOP

```
354
355     while (!WindowShouldClose())
356     {
357         //Event handling
358         if(IsKeyPressed(KEY_UP)){
359             Input_Enqueue(&Input, (Vector2){0, -1});
360             running = true;
361             paused = false;
362         }
363         if(IsKeyPressed(KEY_DOWN)){
364             Input_Enqueue(&Input, (Vector2){0, 1});
365             running = true;
366             paused = false;
367         }
368             if(IsKeyPressed(KEY_LEFT)){
369             Input_Enqueue(&Input, (Vector2){-1, 0});
370             running = true;
371             paused = false;
372         }
373         if(IsKeyPressed(KEY_RIGHT)){
374             Input_Enqueue(&Input, (Vector2){1, 0});
375             running = true;
376             paused = false;
377         }
378         if(IsKeyPressed(KEY_P)){
379             paused = true;
380         }
381         if(IsKeyPressed(KEY_SPACE)){
382             PlaySound(wallWarning);
383         }
384
```

EVENT HANDLING

- This Section handles player Input
- Everytime a player presses a key, the game will run when the game is over or it’s paused.
- Input is stored in a queue for smooth movement

THE MAINLOOP
UPDATE LOOP

```
// Update Positions
if(running){
    if(!paused){
        UpdateMusicStream(bgm);
        if(eventTrigger(gameInterval)){
        if(Input.size > 0){
            Vector2 next = Input_Dequeue(&Input);
            if(reversed){
                next.x *= -1;
                next.y *= -1;
            }
            if(!(next.x == -snake.direction.x && next.y == -snake.direction.y)){
                snake.direction = next;
                running = true;
            }
        }
        if(speedTimer > 0){
            speedTimer--;
            if(speedTimer == 0){
                gameInterval = 0.2;
            }
        }
        if(scoreMultiTimer > 0){
            scoreMultiTimer--;
            if(scoreMultiTimer == 0){
                scoreMulti = 1;
            }
        }
        if(reversedTimer > 0){
            reversedTimer--;
            if(reversedTimer == 0){
                reversed = false;
                snake.direction.x *= -1;
                snake.direction.y *= -1;
                Init_Input(&Input);
            }
        }
```

```
            struct FoodNode* eaten = Check_Food_Collision_V2(head, &snake);
            if(eaten){
                Apply_Food_Effect(eaten, &snake, &textpop);
                Link_Dele_Node(&head, eaten);
                SpawnFood(&head, &snake, &wall);
            }
            MoveSnakeUpdate(&snake);
            WallCollisionCheck(&snake, &head, &wall);
            Check_Snake_Wall_Collision(&wall, &snake, &head);
        }
        Wall_Flash_Update(&wall);
        UpdateTextPop(&textpop);
        }
    }
```

- Handles things such as timers, collisons, animations and etc.

THE MAINLOOP
DRAW

```
439         // Draw
440         BeginDrawing();
441             ClearBackground(Background);
442             DrawRectangleLinesEx((Rectangle){(float)OFFSET-5, (float)OFFSET+70 + (CELL_SIZE * 0.4), (float)CELL_SIZE*CELL_COUNT+10, (float)CELL_SIZE*CELL_COUNT-62 - (CELL_SIZE*0.15)}, 5,darkGreen);
443             Draw_Food(head);
444             DrawSnake(&snake);
445             int scoreWidth = MeasureText(TextFormat("%i", snake.score), 20);
446             DrawText(TextFormat("%i", snake.score), (2*OFFSET + CELL_SIZE*CELL_COUNT) - scoreWidth - 25, 40, 40, YELLOW);
447             Draw_Food_Effect_Text();
448             DrawTextPop(&textpop, &snake);
449             AnimGameOver(&textpop);
450             Draw_Wall(&wall);
451             Game_Paused();
452         EndDrawing();
```

- Handles all the drawings such as the food, score, snake and etc.

```
453         }
454         UnloadMusicStream(bgm);
455         UnloadSound(eatSFX);
456         UnloadSound(crashSFX);
457         UnloadSound(slowSFX);
458         UnloadSound(fastSFX);
459         UnloadSound(wallWarning);
460         CloseAudioDevice();
461         CloseWindow();
462         return 0;
463     }
```

- Unloads all music and closes the window when the window is closed.

CONCEPTS USED IN THE SYSTEM

Data Structure	Purpose
Deque	Snake Movement
Queue	Input Handling
Linked List	Food Management

Each data structure was selected because it best solves a specific problem inside the game.

DEQUE — SNAKE MOVEMENT SYSTEM

A Deque (Double-Ended Queue) is a data structure that allows insertion and deletion from both the front and rear.

In our system, deque is used to manage the movement of the snake body.

WHY WE USE DEQUE ?

Every movement of the snake performs two operations:

- Insert a new head
- Delete the old tail

```
Before:                    After:
[Head] [Body] [Tail]       [New Head] [Head] [Body]
```

```
439     void InsertFront(Deque* deque, Vector2 pos){
440         if(deque->head == 0){
441             deque->head = CELL_MAX -1;
442         } else{
443             deque->head = deque->head -1;
444         }
445         deque->segment[deque->head] = pos;
446         deque->size++;
447     }
448
449     void InsertRear(Deque* deque, Vector2 pos){
450         if(deque->tail == CELL_MAX - 1){
451             deque->tail = 0;
452         } else {
453             deque->tail = deque->tail + 1;
454         }
455         deque->segment[deque->tail] = pos;
456         deque->size++;
457     }
458
459     void DeleteRear(Deque* deque){
460         if(deque->tail == 0){
461             deque->tail = CELL_MAX-1;
462         } else{
463             deque->tail = deque->tail-1;
464         }
465         deque->size--;
466     }
467
468     void DeleteFront(Deque* deque){
469         if(deque->head == CELL_MAX -1){
470             deque->head = 0;
471         } else{
472             deque->head = deque->head +1;
473         }
474         deque->size--;
```

SNAKE DEQUE

- InsertFront() and InsertRear() add new segments to the snake, while DeleteFront() and DeleteRear() remove segments during movement.
- The code uses C**ircular Array** logic so the snake can continuously move without running out of memory space.

```
478     //Draw Snake and Movement start--------------------------
479     void DrawSnake(Snake* snake){
480         for(int i = 0; i < snake->body.size; i++){
481             int index = (snake->body.head + i) % CELL_MAX;
482             float x = snake->body.segment[index].x;
483             float y = snake->body.segment[index].y;
484             Rectangle snakeBody = (Rectangle){OFFSET+ x*cellSize, OFFSET+60 + y*cellSize, (float)cellSize, (float)cellSize};
485             DrawRectangleRounded(snakeBody, 0.5, 6, WHITE);
486         }
487
```

- DrawSnake() is responsible for displaying the snake on the screen. It loops through all snake segments, calculates their positions, and draws the snake body and eyes based on the snake’s current direction.

CIRCULAR ARRAY

```
int index = (snake->body.head + i) % CELL_MAX;
```

- The array is circular so the snake can keep moving without running out of space in the array.
- When the head reaches the last index, it wraps around back to index "0", allowing freed spaces from "DeleteRear()" to be reused.

EXAMPLE

```
head = 624
size = 3

i = 0 : (624 + 0) % 625 = 624 → segment[624] = Head
i = 1 : (624 + 1) % 625 = 0   → segment[0] = Body
i = 2 : (624 + 2) % 625 = 1   → segment[1] = Tail
```

- The snake head is currently at index 624, which is the last position of the array. Normally, the next position would become 625, but the array only goes from 0 to 624.
- Using the modulo operator % 625, the next index automatically wraps back to 0, so the snake can continue storing body segments at the beginning of the array.
- This makes the array circular, allowing the snake to move continuously while reusing freed spaces from DeleteRear() instead of running out of memory.

```
void MoveSnakeUpdate(Snake* snake){
    if(reversed){
        if(snake->growSnake > 0){
            InsertRear(&snake->body, Vector2Subtract(snake->body.segment[snake->body.tail], snake->direction));
            snake->growSnake--;
        } else{
            InsertRear(&snake->body, Vector2Subtract(snake->body.segment[snake->body.tail], snake->direction));
            DeleteFront(&snake->body);
        }
    } else{
        if(snake->growSnake > 0){
            InsertFront(&snake->body, Vector2Add(snake->body.segment[snake->body.head], snake->direction));
            snake->growSnake--;
        } else{
            InsertFront(&snake->body, Vector2Add(snake->body.segment[snake->body.head], snake->direction));
            DeleteRear(&snake->body);
        }
    }
}
//Draw Snake and Movement end--------------------------
```

- MoveSnakeUpdate() controls the snake movement by adding a new head in the movement direction and removing the old tail. If the snake eats food, it skips deleting the tail so the snake grows longer.

QUEUE — INPUT HANDLING SYSTEM

A Queue follows the FIFO principle:
First In, First Out
The first input entered is the first processed.

WHY WE USE QUEUE ?

Without queue:

- fast player inputs may disappear

Example: `RIGHT → UP → LEFT`

The game may only detect the last input.

With queue:

- all inputs are stored and processed in order

INPUT QUEUE

```
89  typedef struct{
90      Vector2 sinput[INPUT_QUEUE_SIZE];
91      int front;
92      int rear;
93      int size;
94  }InputSnake;
95  InputSnake Input;
```

- sinput[] — an array of directions
- front — the oldest input
- rear — the next new input will be placed
- size — how many currently queued

WHY DO WE EVEN NEED THIS?

Without the Input queue the controls will be delayed and only response with the last key, making the movement feel slow and unresponsive. The queue stores the inputs so no lost keys between the ticks.

INPUT QUEUE

```
368  //------------------------------------------------------------------
369
370  void Init_Input(InputSnake* input){
371      input->front = 0;
372      input->rear = 0;
373      input->size = 0;
374  }
375
376  void Input_Enqueue(InputSnake* input, Vector2 point){
377      if(input->size >= INPUT_QUEUE_SIZE){
378          return;
379      }
380      input->sinput[input->rear] = point;
381      input->rear = (input->rear + 1) % INPUT_QUEUE_SIZE;
382      input->size++;
383  }
384
385  Vector2 Input_Dequeue(InputSnake* input){
386      if(input->size <= 0){
387          return (Vector2){0,0};
388      }
389      Vector2 point = input->sinput[input->front];
390      input->front = (input->front + 1) % INPUT_QUEUE_SIZE;
391      input->size--;
392      return point;
393  }
```

- Everytime the player clicked a direction key(wasd) the input is positioned at the back of the line, like a waiting line.
- The snake is moved by the game using the oldest input from the front of the line at each game tick this guarantees no player keystroke are overlooked.

LINKED LIST — FOOD MANAGEMENT SYSTEM

A Linked List is a collection of connected nodes.

WHY WE USE LINKED LIST ?

The food system constantly:

- adds food
- removes food

Using arrays would require shifting elements.

Linked list only changes pointers.

```
Example:                          Delete Food2:
Food1 → Food2 → Food3              Food1 → Food3
```

THE FOOD SYSTEM

```
139  typedef struct FoodNode{
140      Vector2 position;
141      FoodType type;
142      struct FoodNode* next;
143  }FoodNode;
144
```

- We 1st initialize the variables

```
108  //Food Structs_____________________________________________________________
109  typedef enum{          //Description:       //Color:     //Chance:
110      NormalFood,        // The generic food  // Red        //Very Common
111      ZaApplo,           // 0.8x speed        // Yellow     //Epic
112      FoodInHeaven,      // 2x speed          // White      //Uncommon
113      BigApple,          // +3 segments       // Dark Red   //Uncommon
114      Minimize,          // -2 segments       // Purple     //Epic
115      FoodRush,          // 2x points         // Orange     //Epic
116      Reverse            // head on tail      //Gray        //Epic
117  }FoodType;
118
```

THE FOOD SYSTEM

```
710  //Linked List Start----------------------------------------
711  struct FoodNode* CreateFoodNode(Vector2 position, FoodType type){
712      struct FoodNode* newNode = (struct FoodNode*)malloc(sizeof(struct FoodNode));
713      newNode->position = position;
714      newNode->type = type;
715      newNode->next = NULL;
716      return newNode;
717  }
718
```

Before Creating a new node:
Create node not called
Nodes: [Node 1 and all struct variables] -> Null

During creating a new node:
Create node called and is getting an empty space in memory
Nodes: [Node 1 and all struct variables] -> [Empty Space/Memory]-> Null

Filling up the empty memory with the variables
Nodes: [Node 1 and all struct variables] -> [Filling in Empty Space]-> Null

Then we return the address of the newNode in that empty space. Meaning we tell the Node 1 where Node 2 is.

After creating a new node:
Nodes: [Node 1 and all struct variables] -> [Node 2 and all struct variables]-> Null

THE FOOD SYSTEM

```
716     return newNode;
717 }
718
719 void Link_Add_Beg(struct FoodNode** head, Vector2 position, FoodType type){
720     struct FoodNode* newNode = CreateFoodNode(position, type);
721     newNode->next = *head;
722     *head = newNode;
723 }
```

- This is the function that creates a food node in the beginning of the list. We put the current head or start of the list in the next node.

Before Creating
Nodes: [Node1] -> Null

We put the current value of head in the nextnode:
Nodes: [Currently Empty] -> [Node1] -> Null

In line 658 we make the head have the newnode variables:
Nodes: [NewNode: Different values] -> [Node1: Different values] -> Null

THE FOOD SYSTEM

```c
void Link_Dele_Node(struct FoodNode** head, struct FoodNode* target){
    struct FoodNode* walker = *head;
    if(*head == NULL){
        return;
    }
    if(*head == target){
        *head = target->next;
        free(target);
        return;
    }

    while(walker->next !=NULL && walker->next != target){
        walker = walker->next;
    }

    walker->next = target->next;
    free(target);
}
```

- If list is empty return
- If the head is the target, move head forward, free the target and return.
- Traverse until walker.next finds the target, replace walker.next with the targets neighbor. Then free the target.

Example:

```
Before: [Node1] -> [Node2] -> [Target] -> [Node4] -> NULL
After:  [Node1] -> [Node2] -> [Node4] -> NULL
```

THE FOOD SYSTEM

```c
void Link_Clear(struct FoodNode** head){
    struct FoodNode* walker = *head;
    while(walker != NULL){
        struct FoodNode* temp = walker->next;
        free(walker);
        walker = temp;
    }
    *head = NULL;
}
```

temp: [Empty]
Nodes: [walker: Node1]-> [walker.next: Node2] -> [Node3]-> Null

Store walker.next in temp and free walker:
temp = Node2
Nodes: []->[Node2] -> [Node3]-> Null

Then:
Nodes: [Node2] -> [Node3]-> Null

THE FOOD SYSTEM

```
void SpawnFood(struct FoodNode** head, Snake* snake, WallPattern* wall){
    Vector2 position = Random_Food_Position();
    while(CheckFoodInSegment(snake, position, 0) || Check_Food_In_Wall(wall, position, 0) || Check_Food_In_Food(*head, position)){
        position = Random_Food_Position();
    }
    FoodType type = FoodTypeChance();
    Link_Add_Beg(head, position, type);
}
```

- Get a Random position in the map

```
Vector2 Random_Food_Position(){
    return (Vector2){GetRandomValue(0, CELL_COUNT-1),
        GetRandomValue(1, PLAY_ROWS-1)};
}
```

- Check if the food is inside the snake

```
bool CheckFoodInSegment(Snake* snake, Vector2 point, int index){
    for(unsigned int i = index; i < snake->body.size; i++ ){
        int bodyIndex = (snake->body.head + i) % CELL_MAX;
        if(Vector2Equals(snake->body.segment[bodyIndex], point)){
            return true;
        }
    }
    return false;
}
```

- Check if food is in the wall

```
bool Check_Food_In_Wall(WallPattern* wall, Vector2 point, int index){
    for(unsigned int i = index; i < wall->count; i++){
        if(Vector2Equals(point, wall->cell[i])){
            return true;
        }
    }
    return false;
}
```

- Check if food is inside itself

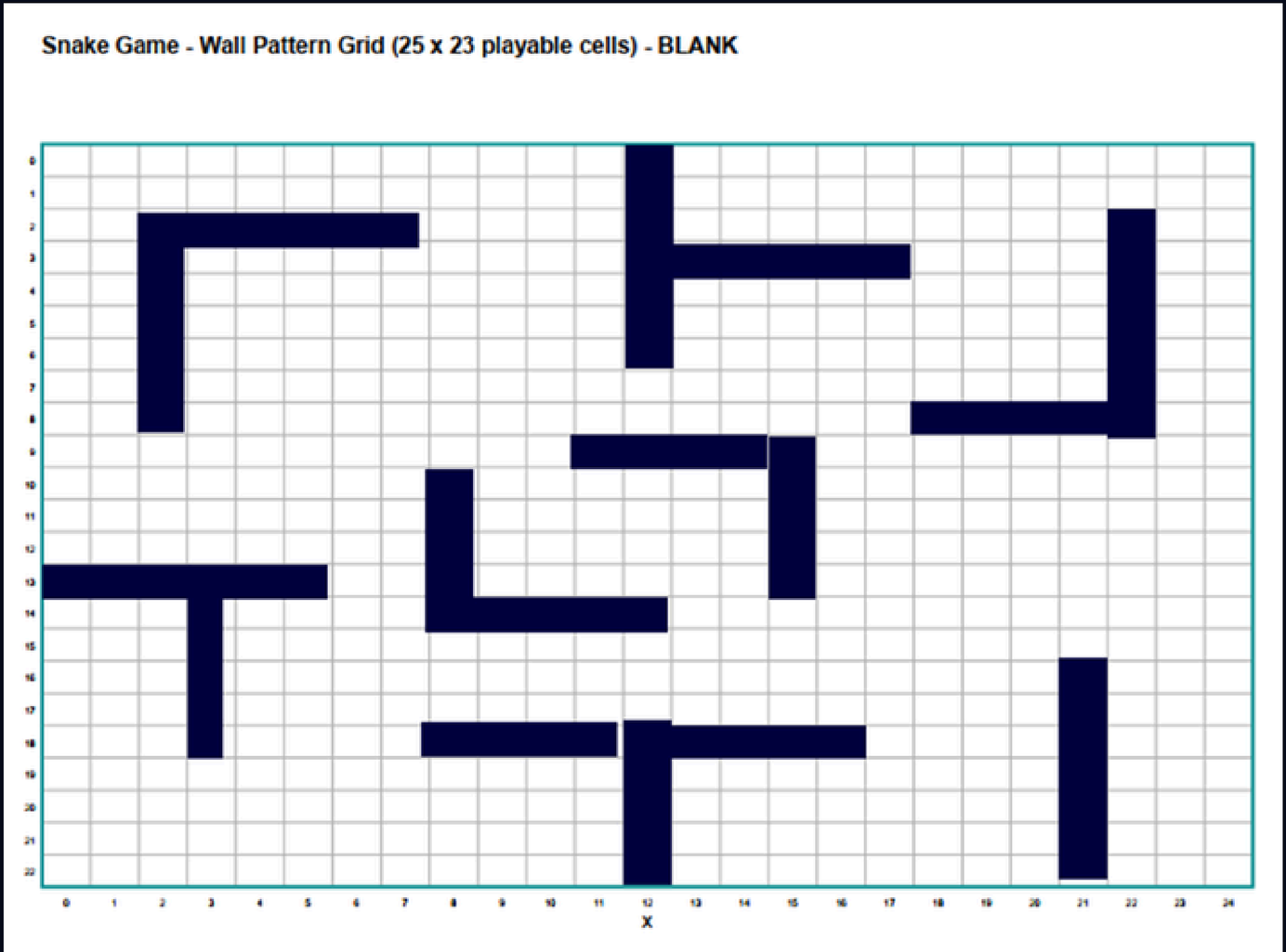
```
bool Check_Food_In_Food(FoodNode* head, Vector2 point){
    struct FoodNode* walker = head;
    while(walker != NULL){
        if(Vector2Equals(walker->position, point)){
            return true;
        }
        walker = walker->next;
    }
    return false;
}
```

THE FOOD SYSTEM

```
804  struct FoodNode* Check_Food_Collision_V2(struct FoodNode* head, Snake* snake){
805      struct FoodNode* walker = head;
806      if(reversed){
807          while(walker != NULL){
808              if(Vector2Equals(walker->position, snake->body.segment[snake->body.tail])){
809                  return walker;
810              }
811              walker = walker->next;
812          }
813      } else{
814          while(walker != NULL){
815              if(Vector2Equals(walker->position, snake->body.segment[snake->body.head])){
816                  return walker;
817              }
818              walker = walker->next;
819          }
820      }
821      return NULL;
822  }
```

- Checks food if it’s eaten by the snake.
- It Checks by constantly checking if every food position is equal to the snake head

THE WALL



- Wall is an addition for the arrays and a gimmick to make the game engaging and challenging.
- We initialize the coordinates of the wall. The wall pattern looks like the picture on the left

THE WALL

```
    // wall 9: Right-bottom tall vertical
    wall.cell[74] = (Vector2){21, 15};
    wall.cell[75] = (Vector2){21, 16};
    wall.cell[76] = (Vector2){21, 17};
    wall.cell[77] = (Vector2){21, 18};
    wall.cell[78] = (Vector2){21, 19};
    wall.cell[79] = (Vector2){21, 20};
    wall.cell[80] = (Vector2){21, 21};
    wall.cell[81] = (Vector2){21, 22};

    wall.count = 82;
    wall.activeWalls = 0;
    return wall;
}
```

- Wall.count is for how many walls there are
- Wall.Active is for how many are currently active

```
void Show_Wall(WallPattern* wall){
    if(wall->activeWalls >= wall->count){
        return;
    }

    int actualWalls[] = {
    5,      //wall 1
    15,     //wall 2
    20,     //wall 3
    29,     //wall 4
    40,     //wall 5
    49,     //wall 6
    60,     //wall 7
    73,     //wall 8
    81      //wall 9
    };
    int wallAmmount = 9;
    int currentGroup = 0;

    for(unsigned int i = 0; i < wallAmmount; i++){
        if(wall->activeWalls <= actualWalls[i]){
            pendingWall = actualWalls[i];
            flashWallTimer = 3.0;
            isWallFlashing = true;
            PlaySound(wallWarning);
            return;
        }
    }
}
```

- The for loop checks which wall should it show by checking if the active walls are <= to the current actualWall array.

THE WALL

```
void Draw_Wall(WallPattern* wall){
    if(pendingWall != -1 && isWallFlashing){
        for(unsigned int i = wall->activeWalls; i < pendingWall+1; i++){
            float x = wall->cell[i].x;
            float y = wall->cell[i].y;
            Rectangle r = (Rectangle){OFFSET + x*cellSize, OFFSET+60 + y*cellSize, (float)cellSize, (float)cellSize};
            DrawRectangleRounded(r, 0.3, 4, RED);
        }
    }

    for(int i = 0; i <= wall->activeWalls - 1; i++){
        float x = wall->cell[i].x;
        float y = wall->cell[i].y;
        Rectangle wallPat = (Rectangle){OFFSET+ x*cellSize, OFFSET+60 + y*cellSize, (float)cellSize, (float)cellSize};
        DrawRectangleRounded(wallPat, 0.3, 4, darkGreen);
    }
}
```

- For loops are utilized for the wall flashing and the walls actually being drawn

LOVE YOU SIR🫶!